

## Solution Requirements

|                      |                          |
|----------------------|--------------------------|
| <b>Date</b>          | 02 October 2026          |
| <b>Team ID</b>       | 6ab3ed02b5170a9b58b3aa3a |
| <b>Project Name</b>  | ComicCraft AI            |
| <b>Maximum Marks</b> | 4 Marks                  |

### Define Functional and Non-Functional Requirements:

Create a comprehensive list of requirements to define exactly what your solution will do and how well it will perform.

- **Functional Requirements (FR)** define specific behaviors, functions, or features of the system (What the system should *do*).
- **Non-Functional Requirements (NFR)** specify the criteria that judge the operation of a system, rather than specific behaviors (How the system should *be*).

Fill out the descriptions and necessary details for each category provided below.

### Step 1: Functional Requirements (FR)

| S.No | Requirement Category | Requirement Description                           | Priority (High/Medium/Low) |
|------|----------------------|---------------------------------------------------|----------------------------|
| 1    | Authentication       | No login required for local project use.          | Low                        |
| 2    | Authorization levels | Single-user local workflow; no role-based access. | Low                        |
| 3    | External interfaces  | Gemini API and Pollinations API.                  | High                       |

|                                 |                                   |                                                               |        |
|---------------------------------|-----------------------------------|---------------------------------------------------------------|--------|
| for story and image generation. |                                   |                                                               |        |
| 4                               | Transactions processing           | Generate story, create 5 panels, and render final comic.      | High   |
| 5                               | Reporting                         | Display generated panels and support final comic export.      | Medium |
| 6                               | Business rules, etc.              | Exactly 5 stages; consistent character details across panels. | High   |
| 7                               | Compliance to laws or regulations | Keep API keys private in .env; never commit secrets.          | High   |
| 8                               | Other                             | Allow panel regeneration and support readable comic text.     | Medium |

## Step 2: Non-Functional Requirements (NFR)

| S.No | NFR Category | Requirement Description | Target Metric / Acceptance Criteria |
|------|--------------|-------------------------|-------------------------------------|
|      |              |                         |                                     |

|   |                             |                                                                          |                                                |
|---|-----------------------------|--------------------------------------------------------------------------|------------------------------------------------|
| 1 | Performance & Response Time | Generation should complete without unnecessary delay; API time may vary. | Acceptable response for normal use.            |
| 2 | Scalability                 | Backend structure should support more users and requests.                | Modular API; no fixed small-user limit.        |
| 3 | Security & Data Privacy     | Protect API keys and avoid storing secrets in source code.               | Keys only in .env; secrets absent from GitHub. |
| 4 | Reliability & Availability  | Use fallback image generation when the primary service is unavailable.   | Comic flow continues when possible.            |
| 5 | Usability & Accessibility   | Provide a simple form and readable generated comic.                      | Clear labels and readable text.                |
| 6 | Other                       | Keep code modular, maintainable, and easy to update.                     | Clear project structure and reusable modules.  |